

W08D02 — Deploy FastAPI ke Railway

ML Engineer Journey | Phase 2 — API & Deploy | 28 Apr 2026

1. Apa Itu Deployment — Analogi Toko Roti

Bayangkan kamu punya toko roti di rumah. Kamu sudah tahu cara membuat rotinya, resepnya bagus, dapurnya bersih. Tapi tiba-tiba kamu diminta buka toko di mall — tempat orang lain yang mengelola listrik, air, dan sistem pendinginnya. Di mall itu, kamu tidak bisa bilang 'di rumah saya oven-nya sudah panas' — kamu harus membawa semua instruksi dari nol: oven apa yang dipakai, suhu berapa, bahan apa saja yang dibutuhkan, dan bagaimana cara tahu toko sudah siap buka.

Itulah deployment. Railway.app adalah mall-nya. Kode kamu adalah toko rotinya. Tugas kamu adalah membuat instruksi yang cukup jelas sehingga mall bisa menjalankan toko kamu tanpa kamu ada di sana — dan itulah yang diselesaikan hari ini.

2. Apa Yang Terjadi Saat Deploy

Deployment adalah proses memindahkan aplikasi dari environment lokal ke server yang bisa diakses publik. Railway.app adalah Platform-as-a-Service (PaaS) — mereka mengelola infrastruktur server, kamu hanya perlu menyediakan kode dan instruksi cara menjalankannya. Setiap git push ke branch yang terhubung akan memicu deploy ulang secara otomatis.

Urutan fase yang dijalankan Railway saat deploy:

Fase	Durasi	Yang Terjadi	Sering Gagal Karena
Clone	~1s	Railway clone repo dari GitHub	Branch salah / repo private
Install deps	~30-120s	pip install -r requirements.txt	Package tidak ada / versi konflik
Start app	~2-5s	Jalankan start command	Import error / --host tidak 0.0.0.0
Health check	~5-10s	GET /health harus return 200	Model gagal load / endpoint tidak ada
Live	-	URL publik aktif	-

3. File Wajib Sebelum Deploy

Ini adalah checklist toko roti sebelum buka di mall. Tanpa salah satu dari ini, build akan gagal atau app tidak bisa jalan.

requirements.txt — Tulis Manual, Bukan pip freeze

Jangan pernah menggunakan pip freeze langsung untuk generate requirements.txt yang akan di-deploy ke Linux server. pip freeze di Windows akan ikut capture Windows-specific dependencies seperti pywinpty, pywin32, dan colorama yang tidak bisa di-compile di Linux. Railway akan crash saat build dengan error 'failed to build pywinpty'.

Solusinya: tulis requirements.txt secara manual, hanya isi dependency yang benar-benar dibutuhkan aplikasimu:

```
fastapi
```

```
uvicorn
```

```
scikit-learn
```

```
joblib
```

```
python-dotenv
```

```
pydantic
```

```
pydantic-settings
```

```
numpy
```

```
pandas
```

Start Command — --host 0.0.0.0 Wajib

Tanpa --host 0.0.0.0, server hanya listen di localhost (127.0.0.1) di dalam container — Railway tidak bisa mengakses app dari luar container dan deploy dianggap gagal. \$PORT adalah environment variable yang Railway inject otomatis.

```
uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

Health Check Endpoint — Wajib untuk Verifikasi

Railway akan hit endpoint ini setelah startup untuk memverifikasi app hidup. Kalau endpoint tidak ada atau return error, deploy dianggap gagal meskipun app sudah jalan.

```
@app.get("/health")
```

```
def health_check():
```

```
    return {"status": "ok", "version": APP_VERSION}
```

4. Struktur Clean Architecture untuk ML API

Aplikasi yang di-deploy hari ini menggunakan clean architecture dari W07 — setiap lapisan punya tanggung jawab yang jelas dan tidak overlap.

File / Folder	Tanggung Jawab
app/main.py	Entrypoint FastAPI, register router, lifespan startup/shutdown
app/config.py	Baca environment variables — satu tempat, tidak hardcode di mana-mana
app/routers/predict.py	Handle routing dan HTTP — tidak ada business logic di sini
app/schemas/prediction.py	Pydantic schema — validasi input/output, kontrak API
app/services/model_service.py	Load model, jalankan prediksi — semua ML logic ada di sini
app/model/pipeline.joblib	Model terserialisasi hasil training Week 4
requirements.txt	Dependency list — ditulis manual, bukan pip freeze
.env	Environment variables lokal — JANGAN di-push ke GitHub

5. ModelService Singleton — Kenapa Penting

Ini salah satu konsep paling kritis di ML API production. Model machine learning bisa berukuran puluhan hingga ratusan MB. Jika model di-load di dalam fungsi predict(), maka setiap request akan membaca file dari disk, mengalokasikan memori, dan menginisialisasi model dari awal — ini bisa memakan waktu 1-5 detik per request, dan pada traffic tinggi akan membuat server kehabisan memori.

Solusinya adalah Singleton Pattern: buat satu instance ModelService saat aplikasi pertama kali jalan, load model sekali di __init__(), dan gunakan instance yang sama untuk semua request setelahnya. Model hanya pernah di-load sekali selama server hidup.

	Model di dalam predict()	ModelService Singleton
Load model	Setiap request (~1-5 detik)	Sekali saat startup (~1-5 detik total)
Memory usage	Naik terus seiring traffic	Stabil, satu instance
Response time	Lambat (disk I/O tiap request)	Cepat (model sudah di RAM)
Risiko	OOM (Out of Memory) di production	Minimal
Pattern	Anti-pattern	Best practice production

6. Error Umum & Cara Diagnosa

Error di Logs	Penyebab	Solusi
ModuleNotFoundError: No module named xxx	xxx tidak ada di requirements.txt	Tambahkan ke requirements.txt, push ulang
failed to build pywinpty	Package Windows-specific masuk requirements.txt manual tanpa Windows deps	hapus requirements.txt, install deps Windows
Error creating build plan with Railpack	Root Directory belum diset / requirements.txt di Root Directory	Set Root Directory di Railway Settings

Address already in use	Port conflict	Pastikan pakai \$PORT bukan hardcode 8000
Health check failed	/health return error atau timeout	Cek startup logs, pastikan model berhasil load
ModuleNotFoundError: pydantic_settings	pydantic-settings tidak di requirement	Tambahkan pydantic-settings

7. Kekuatan Live Service — Ini Baru Permulaan

Deploy FastAPI dasar hari ini adalah fondasi. Tapi live service yang sudah berjalan di Railway membuka pintu ke hal-hal yang jauh lebih powerful. Berikut apa yang sebenarnya bisa dibangun di atas infrastruktur yang sama:

A/B Testing Model

Karena API sudah live dengan versioning (`model_version` di response), kamu bisa deploy dua versi model secara bersamaan dan route traffic berdasarkan persentase — misalnya 80% ke model lama, 20% ke model eksperimental. Ini cara perusahaan teknologi besar memvalidasi model baru tanpa risiko rollout penuh.

Monitoring & Drift Detection

Setiap request yang masuk ke `/predict` bisa di-log beserta input features dan prediksinya. Dengan data ini kamu bisa detect data drift — kondisi di mana distribusi input production mulai berbeda dari data training, yang menyebabkan akurasi model turun secara diam-diam. Ini salah satu masalah paling umum dan paling sering tidak ketahuan di ML production.

Integrasi dengan Sistem Lain

Live URL berarti sistem apapun yang bisa mengirim HTTP request bisa menggunakan model kamu — aplikasi mobile, dashboard React, sistem ERP perusahaan, Slack bot, Google Sheets via Apps Script, atau bahkan model ML lain sebagai bagian dari ensemble pipeline yang lebih besar.

Automated Retraining Pipeline

Dengan live API, kamu bisa membangun loop: data baru masuk via endpoint, disimpan ke database, trigger retraining otomatis ketika jumlah data mencapai threshold, model baru di-serialize dan di-push, Railway auto-deploy model terbaru. Ini yang disebut Continuous Training — ML system yang belajar sendiri.

Authentication & Rate Limiting

Endpoint yang sudah live bisa dilindungi dengan API key authentication menggunakan FastAPI dependency injection (sudah dipelajari di W05D03). Rate limiting bisa ditambahkan untuk mencegah abuse — misalnya maksimal 100 request per menit per user. Ini mengubah project latihan menjadi product yang bisa dijual.

Portfolio yang Bisa Didemonstrasikan

Ini mungkin yang paling underrated: kamu sekarang punya URL yang bisa kamu tunjukkan ke siapapun, kapanpun, tanpa perlu menjelaskan cara menjalankan kode secara lokal. Recruiter bisa langsung hit endpoint dan melihat model kamu bekerja secara real-time. Ini beda level dibandingkan kandidat yang hanya punya notebook Jupyter.

8. Railway Free Tier — Yang Perlu Diketahui

Aspek	Detail	Implikasi Praktis
Trial credit	\$5 credit saat daftar	Cukup untuk beberapa hari testing
Sleep policy	Tidak ada auto-sleep	Berbeda dari Render — app tetap hidup
RAM	Shared, ~512MB untuk free	Cukup untuk model scikit-learn kecil-menengah

Persistent disk	Tidak ada di free tier	Jangan write file saat runtime — akan hilang saat restart
Auto-deploy	Setiap push ke main branch	Setiap git push = deploy ulang otomatis
Custom domain	Tersedia gratis	Bisa pakai domain sendiri jika punya

9. Ringkasan — Apa yang Sudah Dicapai Hari Ini

1. Membangun ML API dengan clean architecture (routers / schemas / services) dari nol
2. Memahami kenapa pip freeze berbahaya untuk deployment ke Linux dan cara mengatasinya
3. Men-debug tiga error production secara berurutan: Railpack build fail, Windows deps conflict, pydantic_settings missing
4. Berhasil deploy FastAPI dengan model scikit-learn ke Railway dan mendapatkan live public URL
5. Memverifikasi health check dan predict endpoint berjalan di production dengan probability output
6. Memahami ModelService singleton pattern dan kenapa ini critical untuk ML production

github.com/lolivampire/ML-Project | Next: W08D03 — README Production Polish